

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I JAASIEL JS (192524046) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	
--------------	-----	--	--	--------------------------------	------------------------------	--

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. Scenario: Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.

Task:

- A. Input: A grayscale image (size $M \times N$)
- B. Output: Bounding boxes of detected faces and corresponding recognized names
- C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode Task:

1. Preprocess the image (grayscale normalization, histogram equalization)
2. Detect faces using a sliding window or Viola-Jones detector
3. Extract features using Eigenfaces
4. Compare with database of known faces using Euclidean distance
5. Output bounding boxes with recognized labels

ANSWER:

PSEUDOCODE

BEGIN

INPUT grayscale image of size $M \times N$

STEP 1: Preprocess the image
 Perform grayscale normalization
 Apply histogram equalization

STEP 2: Detect faces
 Use Viola-Jones face detector
 Detect all faces in the image

STEP 3: FOR each detected face
 Extract the face region

Resize the face
Extract features using Eigenfaces

STEP 4: Compare extracted features
Compare with database of known faces
Calculate Euclidean distance

STEP 5: Find the closest matching face
Assign the corresponding name

STEP 6: Draw bounding box around each face
Display the recognized name

OUTPUT bounding boxes and recognized names

END

CODING

```
import cv2
import numpy as np
from sklearn.decomposition import PCA
from skimage import data

# Load sample image
image = cv2.cvtColor(data.astronaut(), cv2.COLOR_RGB2BGR)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Histogram equalization
gray = cv2.equalizeHist(gray)

# Load Haar Cascade face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades +
    "haarcascade_frontalface_default.xml"
)

# Detect face
faces = face_cascade.detectMultiScale(
    gray, 1.1, 5, minSize=(60, 60)
)

# Get detected face
x, y, w, h = max(faces, key=lambda r: r[2] * r[3])
face = gray[y:y+h, x:x+w]
```

```

# Resize face
face = cv2.resize(face, (100, 100))

# Create simple Eigenface database
known_faces = [face, face, face]

X = np.array([f.flatten() / 255.0 for f in known_faces])

# Calculate mean face
mean_face = X.mean(axis=0)

# Eigenfaces using PCA
pca = PCA(n_components=2)
pca.fit(X - mean_face)

# Test face
test_face = face.flatten() / 255.0

# Project into Eigenface space
projected = pca.transform(
    (test_face - mean_face).reshape(1, -1)
)

known_projected = pca.transform(X - mean_face)

# Calculate Euclidean distance
distance = np.linalg.norm(
    known_projected - projected,
    axis=1
)

# Recognize face
if distance.min() < 0.5:
    name = "Person_1"
else:
    name = "Unknown"

# Display result
cv2.rectangle(
    image,
    (x, y),
    (x+w, y+h),
    (0, 255, 0),
    2
)

cv2.putText(
    image,

```

```

    name,
    (x, y-10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (0, 255, 0),
    2
)

cv2.imshow("Eigenfaces Face Recognition", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

OUTPUT

EIGEN FACE RECOGNITION



2. Scenario: Design an algorithm to detect road lanes in a real-time driving video sequence.

Task:

- A. Input: Video frames from a front-facing camera
- B. Output: Lane lines highlighted on each frame
- C. Requirements: Robust to shadows, curves, and partial occlusion

Pseudocode Task:

1. Capture video frame
2. Convert frame to grayscale
3. Apply Gaussian smoothing to reduce noise
4. Perform edge detection (Canny)

5. Apply Hough Transform to detect straight/curved lines
6. Overlay detected lanes on the original frame
7. Repeat for all frames

ANSWER:

PSEUDOCODE

BEGIN

1. Capture video frame from the front-facing camera.
2. Convert the video frame to grayscale.
3. Apply Gaussian smoothing to reduce noise.
4. Perform Canny edge detection to detect edges.
5. Apply Hough Transform to detect straight and curved lane lines.
6. Overlay the detected lane lines on the original frame.
7. Repeat Steps 1 to 6 for all video frames.
8. Display the output frame with highlighted lane lines.

END

CODING:

```
import cv2
import numpy as np

cap = cv2.VideoCapture("driving.mp4")

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Gaussian smoothing
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```

# Canny edge detection
edges = cv2.Canny(blur, 50, 150)

# Region of Interest
height, width = edges.shape
mask = np.zeros_like(edges)

polygon = np.array([[
    (0, height),
    (width, height),
    (int(width * 0.6), int(height * 0.6)),
    (int(width * 0.4), int(height * 0.6))
]])

cv2.fillPoly(mask, polygon, 255)

roi = cv2.bitwise_and(edges, mask)

# Hough Transform
lines = cv2.HoughLinesP(
    roi,
    1,
    np.pi / 180,
    50,
    minLineLength=50,
    maxLineGap=50
)

# Draw detected lane lines
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]

        cv2.line(
            frame,
            (x1, y1),
            (x2, y2),
            (0, 0, 255),
            4
        )

# Display output
cv2.imshow("Road Lane Detection", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

```
cap.release()  
cv2.destroyAllWindows()
```

OUTPUT

Q2 Output - Road Lane Detection

